

Predictive Coding vs. Backpropagation as an Encoder Objective

A go/no-go result for the joint-embedding world model

Project `pc_predictor` · Galaxy10 · LeJEPA objective · 12 September 2026

Result in one line. Under a LeJEPA-style self-supervised objective, predictive coding produces a measurably *higher-dimensional* latent space than backpropagation but a measurably *less discriminative* one — and increasing PC's inference budget makes the deployed encoder monotonically worse, not better. PC as an encoder training rule is a no-go; PC as the *predictor* rule (the configuration the original proposal targets) remains untested.

Scope. This document covers only the Q1-Phase A experiment line: the encoder-only go/no-go test of whether predictive coding produces richer, more useful latent representations than backpropagation under a LeJEPA-style objective (SIGReg + invariance) on Galaxy10. The literature review and the computation-graph exercises documented elsewhere in the project are out of scope here.

1 Objective

The project's original question — does PC give a better gradient than backprop — is settled negative: PC provably converges to the backprop gradient under standard assumptions. The live question is narrower and different in kind: does PC's local, energy-relaxation learning rule leave a different signature on the *representation* it produces, and is that signature more useful for image-patch prediction inside a joint-embedding world model?

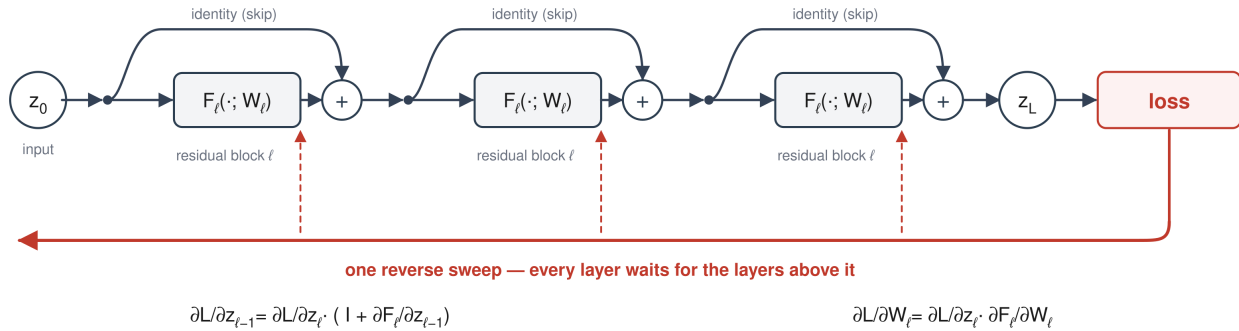
Q1-Phase A is designed as a cheap go/no-go gate before committing to Phase B (a position-conditioned PC predictor on top of a frozen or co-trained encoder). It trains a small residual encoder under a fixed self-supervised loss with the weight update supplied by either backprop or predictive-coding relaxation, and asks whether the resulting embedding is measurably richer or more discriminative under PC.

2 How the two learning rules differ

Both arms minimise the same loss and use the same update rule; they differ only in how the weight gradient is obtained. Backpropagation assembles it in a single global reverse sweep: the gradient at a given layer is a product of Jacobians reaching all the way from the loss, so no layer can update until every layer above it has been traversed. In a residual network each of those Jacobians carries an identity term from the skip connection — the gradient highway that lets ResNets train at depth.

A. Backpropagation — one global reverse sweep

Forward activations are stored, then a single backward pass carries the error from the loss down through every layer.



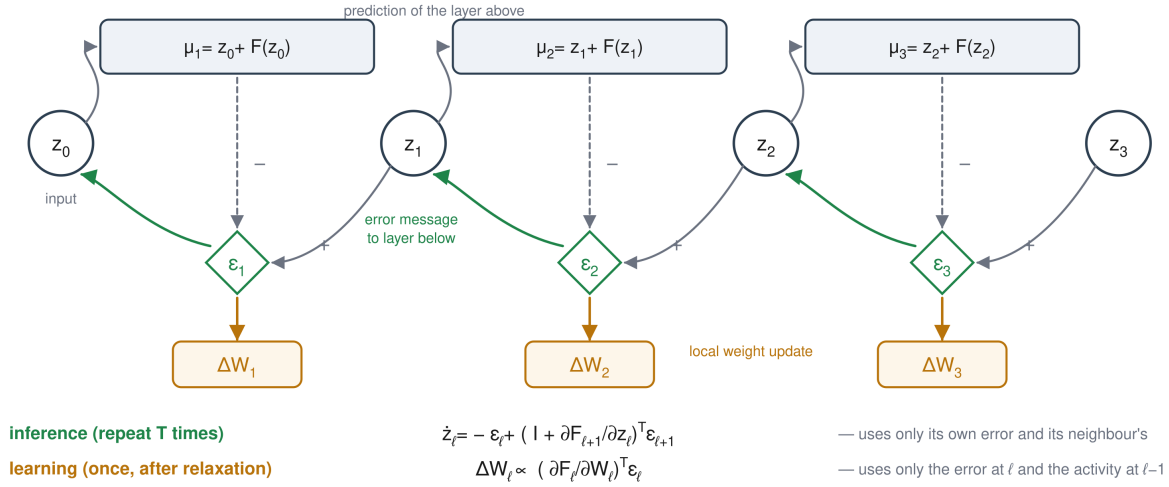
- The 1 term is the skip connection's contribution — the gradient highway that lets ResNets train at depth.
- Non-local: the update for W_l depends on a quantity assembled from every layer above it.
- Exact, but sequential — nothing updates until the sweep reaches the bottom.

Figure 1. Backpropagation on a residual stack. A single reverse sweep carries the error from the loss down through every block; each block's weight gradient is non-local, depending on a quantity assembled from all layers above it. The 1 term is the skip connection's contribution.

Predictive coding replaces that sweep with a relaxation. Every layer holds its own activity z and its own error ϵ , the mismatch between that activity and the prediction made by the layer below. Errors are exchanged only between neighbouring layers, and the exchange is repeated T times before the weights are touched once. Each message uses only quantities local to a layer and its immediate neighbour, so all layers can update in parallel; in a PC-ResNet the skip connection appears again as the same identity term, letting the error message pass downward undamped. At the relaxed fixed point, and under the fixed-prediction assumption, these local updates approximate the backprop gradient.

B. Predictive coding — T steps of local message passing

Every layer carries its own activity and its own error. Errors travel only between neighbouring layers, and they travel repeatedly.



- The skip connection appears again as the I term — in a PC-ResNet the error message also passes downward undamped.
- Local: no layer needs a quantity from further up the graph, so all layers can update in parallel.
- At the relaxed fixed point, and under the fixed-prediction assumption, these updates approximate the backprop gradient.

Figure 2. Predictive coding on the same stack. T steps of local message passing between activity and error nodes replace the global sweep; the weight update fires once, after relaxation, and reads only its own layer's error and the activity below it.

Why this matters for the experiment. The relaxation gives PC a second set of free variables — the activities — that can lower the objective without the weights changing at all. Backprop has no such freedom. Section 6 shows this is not a theoretical curiosity: it is the mechanism behind the result.

3 Experimental setup

Architecture: fully-connected μ PC-style residual MLP, width 128, depth 8, no bias, no normalisation layers.

Data: Galaxy10, images flattened to a 3072-vector, 4 augmented views per image (crop 0.4–1.0, rotation, horizontal flip, brightness/contrast jitter). **Objective:** the LeJEPa recipe — a SIGReg (Epps-Pulley) distributional regulariser against collapse, combined with a cross-view invariance term, $lejepa_loss = \lambda \cdot \text{SIGReg} + (1-\lambda) \cdot \text{invariance}$, $\lambda = 0.05$.

Arms: backprop+SIGReg, backprop invariance-only (control), PC ($T=16$)+SIGReg, PC ($T=16$) invariance-only (control). PC's weight gradient comes from a relaxed activity trace (T inference steps per forward pass); the encoder is evaluated feedforward (E. forward) in both arms, matching how a joint-embedding encoder is actually deployed.

Metrics: a k -NN probe accuracy on the embedding (10 classes, chance = 0.10), effective rank, TwoNN intrinsic dimension, dimensions needed for 95% variance, and the best k -NN probe achievable at any hidden layer.

4 Finding 1 — a collapse confound, diagnosed and fixed

The first version of this grid appeared to show PC beating backprop. Diagnosis traced this to a bug, not a PC effect: the backprop arm's embedding was sitting in an exact, unescapable fixed point of the SIGReg statistic (collapsed to zero), which no learning-rate change could leave, because the SIGReg gradient at exact collapse is identically zero.

Root cause: the invariance-loss reduction summed over the embedding's 128 feature dimensions instead of averaging over them, over-weighting that term by a factor of $128\times$ relative to the LeJEPA reference recipe, and SIGReg saw the raw encoder output rather than a normalising projector. PC's relaxation happened not to fall into this trap; backprop did — a fact about optimisation dynamics on a broken loss, not about representation richness.

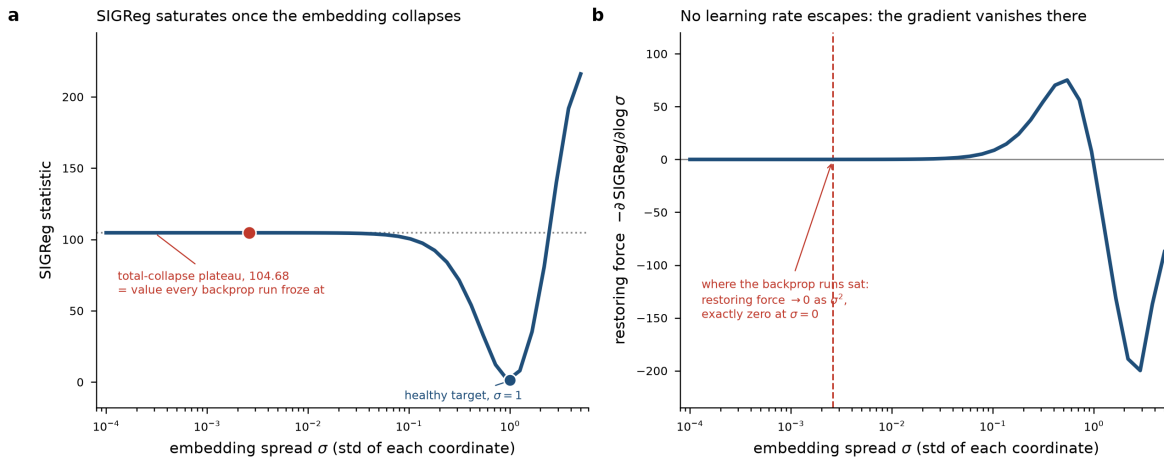


Figure 3. The SIGReg statistic and its gradient near and at exact embedding collapse. The gradient vanishes identically at the collapsed point (104.6837), explaining why no learning rate could recover the pre-fix backprop arm.

Three fixes were applied: (1) average rather than sum the invariance loss over feature dimensions; (2) add a parameterless BatchNorm projector ahead of both loss terms, matching the reference recipe and making the collapsed state unreachable rather than merely unfavourable; (3) add the reference's warmup+cosine learning-rate schedule and AdamW weight decay.

Results after the fix (2,000 steps, T = 16 for PC)

Arm	kNN pre	kNN post	eff. rank pre	eff. rank post	twoNN post	dims-95 post	best hidden kNN
BP + SIGReg	0.1335	0.3605	2.07	21.05	20.92	46	0.3985
BP, invariance only	0.1280	0.2905	1.82	1.02	12.74	1	0.3680
PC (T=16) + SIGReg	0.2980	0.3380	21.54	23.14	20.73	41	0.3935
PC (T=16), invariance only	0.2800	0.2970	23.07	1.36	20.66	1	0.3935

Table 1. Post-fix 2,000-step grid. Random-initialisation reference k-NN probe: 0.343 (chance = 0.10).

With the fix, backprop no longer collapses and is now *ahead* of PC on the probe (0.3605 vs. 0.3380) — the pre-fix "PC wins" result does not survive a working baseline. PC still ends with higher effective rank (23.14 vs. 21.05), but that extra rank does not translate into probe accuracy.

This grid's own limitation: probe accuracy at step 0 is already 0.343, and after 2,000 steps backprop reaches only 0.3605 and PC only 0.3380 — almost all of the accuracy is random-feature accuracy, so the BP–PC gap of 0.023 is real but small against a baseline doing most of the work.

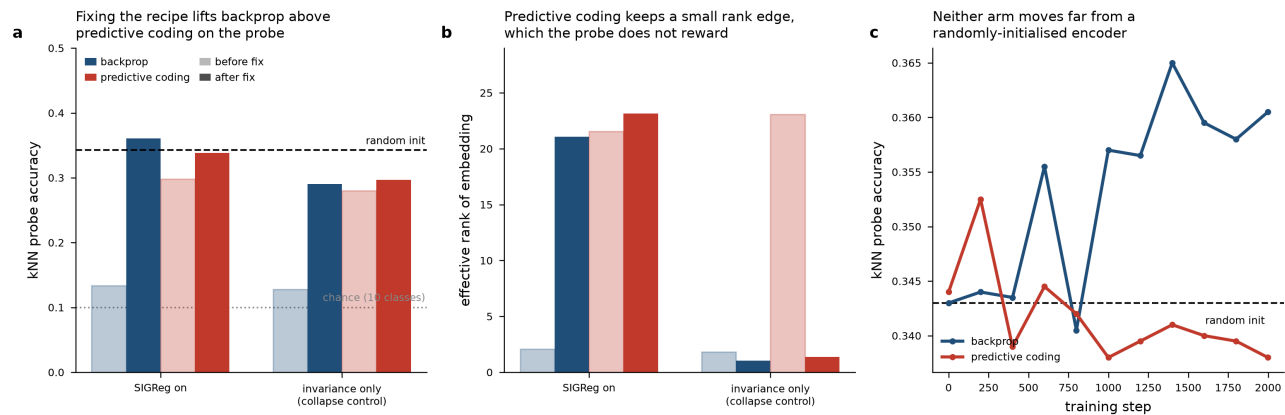


Figure 4. Post-fix 2,000-step grid: probe accuracy, effective rank, and wall-clock across the four arms.

5 Finding 2 — full LeJEPA budget (400 epochs): the gap widens

Same code and fixes, budget raised from 2,000 steps to 17,200 (43 steps/epoch × 400 epochs, matching LeJEPA's own Galaxy10 recipe), evaluated every 20 epochs.

Arm	kNN @2k steps	kNN @400 ep	eff. rank @400 ep	twoNN	dims-95	best hidden kNN	final inv. loss
BP + SIGReg	0.3605	0.3910	18.33	9.22	20	0.4200	0.3999
BP, invariance only	0.2905	0.2655	1.00	7.84	1	0.3855	0.0130
PC (T=16) + SIGReg	0.3380	0.3305	20.81	22.71	50	0.3935	0.6942
PC (T=16), invariance only	0.2970	0.2155	1.00	10.99	1	0.3935	0.0924

Table 2. Full 400-epoch budget, all arms. Random-initialisation reference probe: 0.343.

Backprop keeps learning: +0.048 over its own random-init baseline by 400 epochs, saturating around epoch 80–150. **Predictive coding never exceeds its own initialisation** — it dips to a minimum of 0.3233 around epoch 100 and recovers only to 0.3305, still below the 0.343 random-init floor. The BP–PC gap widened from 0.023 at 2,000 steps to **0.061** at 400 epochs: more budget separated the arms rather than converging them.

PC's final invariance loss (0.6942) is 1.74× higher than backprop's (0.3999) and flat from epoch ~60 — PC is not solving its own training objective as well as backprop is solving theirs, which raised the leading hypothesis at this point: is the T=16 relaxation budget simply too short to produce a useful weight gradient? PC does end with measurably higher effective rank (20.81 vs. 18.33) and over twice the intrinsic dimension (22.71 vs. 9.22) and dimensions-to-95% (50 vs. 20) — "richer" and "better" come apart cleanly: the extra dimensionality is not class-discriminative structure.

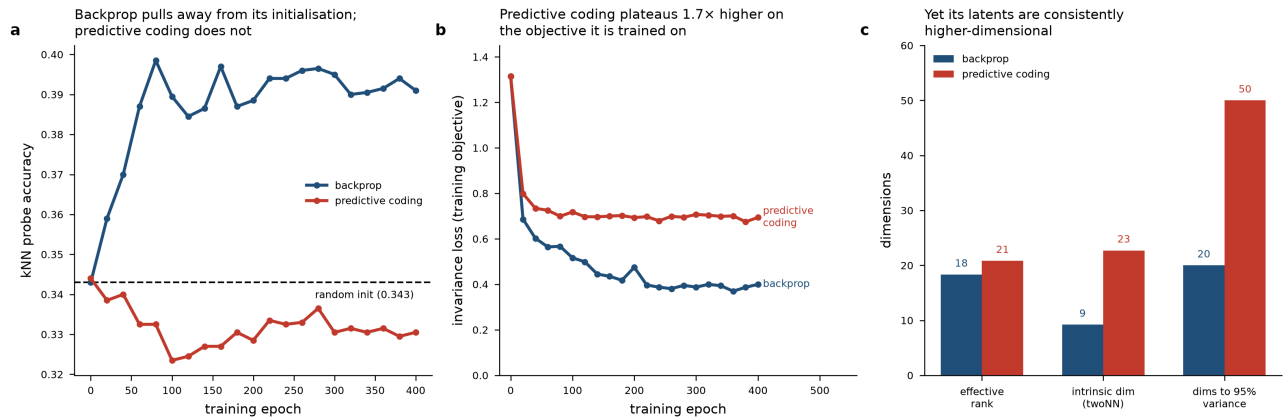


Figure 5. Full 400-epoch budget: probe accuracy trajectories and final representation-geometry metrics per arm.

This motivated the decisive next experiment: a sweep over PC's inference budget T. If more relaxation steps close the invariance-loss gap and raise the probe together, the deficit was under-optimisation and the representation question stays open. If the loss falls but the probe does not follow, PC is producing higher-dimensional but genuinely less discriminative latents.

6 Finding 3 — the T sweep: more inference makes the encoder worse

Same 400-epoch budget, same code, only `cfg.pc.T` varied across 16 / 32 / 64 (T=128 was killed mid-run once the trend was clear; T=256 never started). All reported rows completed the full 17,199-step budget.

T	inv. loss (feedforward)	inv. loss (relaxed)	energy F	kNN probe	eff. rank	TwoNN	dims -95
16	0.6942	0.5744	0.0018	0.3305	20.81	22.71	50
32	0.7469	0.2544	0.0073	0.2970	7.33	19.84	29
64	0.9109	0.0371	0.0075	0.2655	1.71	11.31	2
backprop	0.3999	n/a	n/a	0.3910	18.33	9.22	20

Table 3. Inference-budget sweep. Random-init reference probe: 0.343. The best hidden-layer probe is pinned at 0.3935 across all three T values, unchanged.

Every quantity above is monotone in T, and they move in **opposite directions**: the objective evaluated at the relaxed activities falls 15× (0.5744 → 0.0371 — more relaxation steps solve the task better and better), while the objective evaluated at the feedforward embeddings *rises* (0.6942 → 0.9109), the probe falls (0.3305 → 0.2655, from just below the random-init line to well below it), and the embedding collapses (effective rank 20.81 → 1.71, dimensions-to-95% 50 → 2). Since the evaluation probe reads the feedforward embedding for both arms — the way a joint-embedding encoder is actually deployed — this is a direct penalty on the quantity that matters, not a measurement artifact.

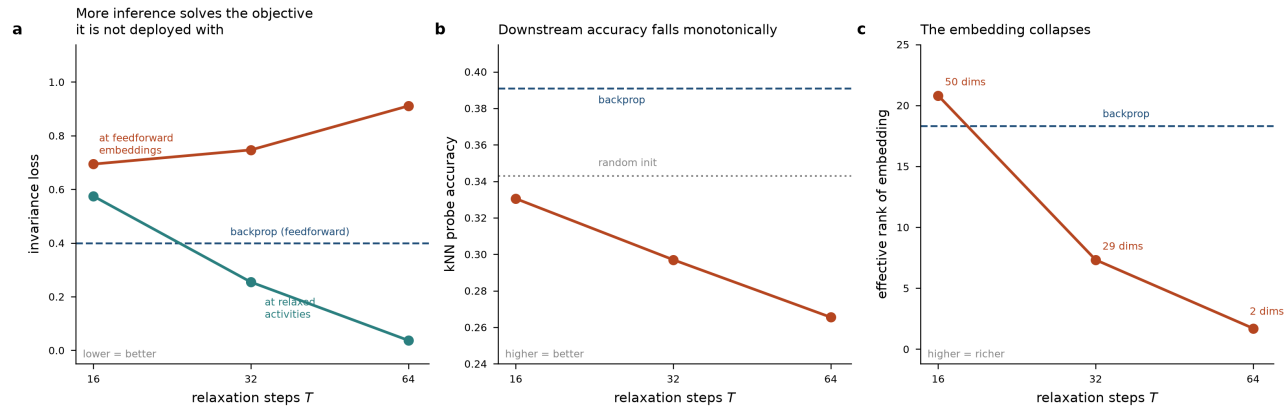


Figure 6. T sweep: relaxed-activity loss, feedforward loss, probe accuracy, and effective rank as functions of inference budget T.

Mechanism

Activity relaxation and the feedforward weights are two independent routes to satisfying the training objective, and they compete for it. With more relaxation steps, the activities absorb more of the objective themselves; the residual driving the weight update shrinks, so the weights are never pressured into a feedforward map that is good on its own. In the limit, relaxation does the work and the feedforward pass drifts — here, toward collapse, because the distributional regulariser's pressure is absorbed by the activities too.

Two routes to lowering the objective — and they compete

Raising T lets the activities absorb more of the objective, leaving less residual to press on the weights.

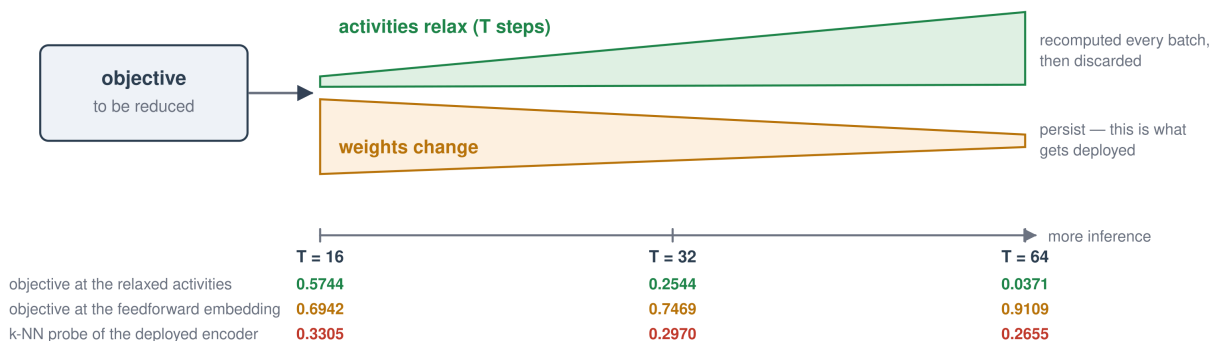


Figure 7. The two routes and how they trade off. Only the weights persist past the batch; as T grows the activities take a larger share of the objective, and the deployed feedforward encoder degrades on every measured axis.

This reverses the previous section's hypothesis. The energy was already converged at $T=16$ ($F = 0.0018$, and not improving with more steps — $F = 0.0075$ at $T=64$), and the best hidden-layer probe stays pinned at 0.3935 regardless of T . Raising T does not fix under-optimisation because there was none to fix: it makes the deployed representation strictly worse.

7 What this settles for the research question

The original question was whether PC yields latents better suited to patch prediction. For **PC used as the encoder's training rule**, this grid gives a clear and now-trustworthy answer: no. Backprop learns a strictly better feedforward representation at every budget tested, the gap widens rather than closes with more compute, and PC's real rank/dimensionality advantage does not convert into discriminative structure — the T sweep shows this is a structural consequence of the learning rule (relaxation and weights compete for the objective), not a matter of running PC longer or harder.

This result is scoped to the encoder. The project's original proposal targets PC for the **predictor** inside a joint-embedding world model, where the output is consumed as a prediction rather than as a deployed embedding — a setting where the relaxation/feedforward decoupling identified here may not apply the same way, or may apply differently. This result is not evidence against that configuration; it closes off "PC as encoder" as a promising direction and sharpens what Phase B needs to test.

8 Not assessed

- Seed variance — every cell in every grid above is $n = 1$.
- T below 16 ($T = 4, 8$) — would test whether the trend is monotone all the way down to very short relaxation.
- Whether the decoupling is specific to the coupled output-drive inference variant, or also holds frozen.
- Whether reading the probe off the relaxed activities (rather than the feedforward pass) recovers accuracy — not the deployment condition for an encoder, but diagnostically interesting.
- Sensitivity to the projector choice (parameterless BatchNorm vs. a learned MLP head).
- Whether backprop's advantage survives a stronger augmentation stack or a convolutional encoder.

9 Next steps

- Move to Phase B: test PC as the training rule for the **predictor** on top of the (backprop-trained) encoder, which is the configuration the original proposal actually targets.
- Run the glimpse/saccade frontend as a validation control that this pipeline can reach published-looking numbers — it changes the input distribution, not the learning rule, so it does not by itself address the decoupling found here.
- If the predictor result also comes back negative for PC, revisit whether a richer-latent effect could still exist under a different objective (e.g. without SIGReg's distributional pressure) or a different relaxation schedule (frozen vs. coupled).